



Pratik Tarifler

AI Destekli Akıllı Mutfak Asistanı

TEKNİK DOKÜMANTASYON

Mimari · Kod Yapısı · İçerik Üretim Pipeline'ları

Görsel Batch İşleme Sistemi

İçindekiler

1 · Proje Genel Bakış	3
1.1 Vizyon ve Problem Tanımı	3
1.2 Proje İstatistikleri	3
2 · Sistem Mimarisi	4
2.1 Katmanlı Mimari	4
2.2 Teknoloji Yığını	5
3 · İçerik Veritabanı (Faz 2)	6
4 · Firebase Backend (Faz 3.1)	7
4.1 Firestore Şeması	7
4.2 Cloud Functions	8
5 · Görsel Üretim Pipeline'ı (Faz 3.2)	9
5.1 Hibrit Kaynak Stratejisi	9
5.2 Batch İşleme Sistemi	10
5.3 Maliyet Analizi	11
6 · Çeviri Pipeline'ı (Faz 3.3)	12
7 · Mobil Uygulama (Faz 4)	13
7.1 3 Modlu UX Mimarisi	13
7.2 Öneri Algoritması	14
8 · Abonelik Sistemi (Faz 5)	15
9 · Dağıtım ve Operasyon	16



Bu Doküman Hakkında

Bu teknik doküman, Pratik Tarifler projesinin tüm mühendislik bileşenlerini kapsar: sistem mimarisi, veritabanı tasarımı, içerik üretim otomasyonu ve özellikle **görsel içeriğin batch (toplu) işleme ile nasıl üretildiği**. Yeni katılan geliştiriciler ve teknik paydaşlar için referans niteliğindedir.

BÖLÜM 1

Proje Genel Bakış

1.1 Vizyon ve Problem Tanımı

Pratik Tarifler, kullanıcının evindeki mevcut malzemelerden yola çıkarak yapay zeka ile tarif öneren bir mobil uygulamadır. Çözdüğü temel problem şudur: her gün milyonlarca insan "akşam ne pişirsem?" sorusuyla karşılaşır ve bu karar yorgunluğu hem zaman kaybına hem de gıda israfına yol açar.

Uygulamanın ayırt edici özelliği **3 modlu keşif sistemidir**. Kullanıcı, niyetine göre farklı bir mod seçer:

- Mod 1 — Evdeki Kalanlar:** Buzdolabı fotoğrafı çekilir, AI malzemeleri tanır, sadece tamamı elde olan tarifler önerilir.
- Mod 2 — Hızlı Tedarik:** Eldeki malzemelere ek 1-3 malzemeye kadar izin verilir, eksikler ve tahmini fiyatlar gösterilir.
- Mod 3 — Sınırsız Keşif:** 2500 tariflik havuzda filtre tabanlı arama.

Teknik Hedefler

Düşük gecikme (öneri <1sn), ölçeklenebilir backend (10K+ MAU), çevrimdışı destek, 13 dil, %99.5+ çökmesiz oturum oranı.

1.2 Proje İstatistikleri

2.500

Küratörlü Tarif

14

Dünya Mutfağı

13

Desteklenen Dil

12.2K

TypeScript Satırı

66

Uygulama Dosyası

17

Ekran

11

Cloud Function

~\$50

Aylık Backend (10K MAU)

Proje 7 faza bölünmüştür. Bu doküman Faz 2-6 arasındaki tüm teknik bileşenleri detaylandırır. Tarif veritabanı 50 tematik batch hâlinde üretilmiş, sıfır duplikasyon ile doğrulanmıştır.

Faz	Kapsam	Durum
Faz 2	2500 TR tarif veritabanı	Tamamlandı
Faz 3.1	Firebase backend (şema, kurallar, indeksler)	Tamamlandı
Faz 3.2	Görsel üretim pipeline'ı	Scriptler hazır
Faz 3.3	Çeviri pipeline'ı (12 dil)	Scriptler hazır

Faz 4	3 modlu mobil uygulama (React Native)	Tamamlandı
Faz 5	Abonelik sistemi (RevenueCat + Stripe)	Tamamlandı
Faz 6	Lansman ve pazarlama hazırlığı	Tamamlandı

BÖLÜM 2

Sistem Mimarisi

2.1 Katmanlı Mimari

Sistem dört ana katmandan oluşur. Veri akışı yukarıdan aşağıya doğrudur; istemci katmanı asla doğrudan veritabanına yazmaz — tüm yazma işlemleri Cloud Functions üzerinden geçer.

① İSTEMCİ KATMANI — React Native + Expo

17 ekran · 10 custom hook · 5 paylaşılan component · i18n (13 dil) · Çevrimdışı SQLite önbellek · Push bildirim

▼ HTTPS / Firebase SDK

② API KATMANI — Cloud Functions (Node.js / TypeScript)

getRecipeRecommendations (3 mod) · analyzePantryPhoto · scanProductLabel · similarRecipes · subscription webhook'ları · oran sınırlama

▼ Admin SDK

③ VERİ KATMANI — Firebase

Firestore (recipes_{lang}, users, ratings) · Cloud Storage (görseller) · Authentication · 13 bileşik indeks

▼ REST / SDK

④ HARİCİ SERVİSLER

Google Vision API (görsel tanıma) · OpenAI (embedding) · RevenueCat (IAP) · Stripe (web ödeme) · Unsplash/Pexels/Pixabay (görsel kaynak)

Güvenlik Prensibi

İstemci katmanı Firestore'a yalnızca **okuma** erişimine sahiptir. Tüm yazma işlemleri (favori, puan, abonelik) Cloud Functions üzerinden doğrulanır. Bu, veri bütünlüğünü garanti eder ve kötüye kullanımı engeller.

2.2 Teknoloji Yığını

Katman	Teknoloji	Gerekçe
Mobil framework	React Native 0.74 + Expo SDK 54	Tek kod tabanı, iOS+Android
Dil	TypeScript	Tip güvenliği, ölçeklenebilirlik
Navigasyon	React Navigation	Stack + Tab + Modal yapısı
Liste performansı	FlashList	Binlerce tarif akıcı kaydırma
Backend	Firebase (Firestore + Functions)	Sunucusuz, otomatik ölçekleme

Görsel tanıma	Google Cloud Vision	Label + object detection
Abonelik	RevenueCat + Stripe	iOS/Android IAP + web ödeme
Çevrimdışı	expo-sqlite	Tarif önbellegei, senkron kuyruęu
Görsel optimizasyon	WebP + BlurHash	~150KB/görsel, progressif yükleme

BÖLÜM 3

İçerik Veritabanı (Faz 2)

Veritabanı, projenin temelidir. 2500 tarif **50 tematik batch** hâlinde üretilmiş, her batch ayrı bir JSON dosyasında tutulmuştur (örn. `01_soups_complete.json`). Bu modüler yapı, kalite kontrolü ve aşamalı genişlemeyi kolaylaştırır.

3.1 Veri Dağılımı

Boyut	Dağılım
Mutfak	Türk 1077 · Diğer 431 · İtalyan 259 · Akdeniz 233 · +10 dünya mutfağı
Zorluk	Kolay 1606 (%64) · Orta 752 · Zor 142
Diyet	Vejetaryen 1207 · Glutensiz 937 · Vegan 443
Erişim	Ücretsiz 1998 · Premium 502
Hız	Hızlı (≤15dk) 295 · Pratik az malzeme 123

3.2 Tarif Veri Şeması

Her tarif aşağıdaki yapıda saklanır. `ingredient_tokens` alanı, öneri algoritmasının kalbidir — normalize edilmiş malzeme kimlikleridir.

```
// Recipe nesnesi (TypeScript tipi)
{
  id: "tr-mercimek-corbasi",
  title: "Mercimek Çorbası",
  cuisine: "turkish",
  difficulty: "easy",
  total_time_min: 35,
  servings: 4,
  ingredient_tokens: ["lentil", "onion", "carrot", "flour"],
  ingredients: [{ name: "kırmızı mercimek", amount: 1, unit: "cup" }, ...],
  steps: [{ order: 1, text: "Soğanı kavurun..." }, ...],
  diet_tags: ["vegetarian", "vegan"],
  meal_type: ["soup", "lunch"],
  is_premium: false,
  image: { full: "...", thumb: "...", blur_hash: "L6Pj..." },
  rating_avg: 4.8, rating_count: 0, view_count: 0
}
```

✓ Token Sistemi Neden Önemli?

Kullanıcı "kırmızı mercimek" yazsa da "mercimek" arasa da aynı `lentil` token'ına eşlenir. Bu normalize katman, Mod 1/2 öneri algoritmasının doğru çalışmasını sağlar — 60+ malzeme token'ı tanımlıdır.

3.3 Çıktı Formatları

Dosya	Boyut	Kullanım
recipes_db_FULL_tr_v2.json	8.8 MB	Ana referans, v2 görsel şeması
recipes_db_firestore_tr.json	5.9 MB	Firestore import formatı
recipes_db_ndjson_tr.ndjson	4.0 MB	Satır bazlı toplu import

BÖLÜM 4

Firestore Backend (Faz 3.1)

4.1 Firestore Koleksiyon Yapısı

Tarifler her dil için ayrı koleksiyonda tutulur (`recipes_tr` , `recipes_en` , ...). Bu, dil bazlı sorgu performansını artırır. Görseller ise dilden bağımsızdır — tüm diller aynı görsel URL'sini paylaşır.

```
// Firestore koleksiyon ağacı
recipes_tr/{recipeId}      // 2500 doküman, dil başına
recipes_en/{recipeId}
...
users/{userId}
  ├── favorites/{recipeId}
  ├── scan_history/{scanId}
  └── quotas/{date}         // günlük scan limiti takibi
ratings/{userId}_{recipeId} // kullanıcı puanları
analytics_events/{eventId}  // olay günlüğü
```

13 Bileşik İndeks

Firestore çoklu-alan sorguları için bileşik indeks gerektirir. Örnek: `cuisine + rating_avg` , `meal_type + total_time_min` , `ingredient_tokens + image_status` . Tümü `firestore.indexes.json` 'da tanımlıdır ve `firebase deploy` ile yüklenir.

4.2 Cloud Functions

11 fonksiyon üç kategoriye ayrılır:

Tip	Fonksiyon	Görev
Callable	getRecipeRecommendations	3 modun ana algoritması
Callable	analyzePantryPhoto	Vision API ile fotoğraf analizi
Callable	scanProductLabel	OCR — ürün etiketi tarama
Callable	getSimilarRecipes	Embedding tabanlı benzer tarif
Callable	submitRating, incrementViewCount	Kullanıcı etkileşimleri
Trigger	onRatingChange	Puan ortalaması yeniden hesabı
Trigger	onFavoriteChange	Favori sayacı senkronizasyonu
Scheduled	cleanupPantryScans	24s+ eski fotoğrafları siler
Scheduled	notifyTrialExpiringSoon	Deneme bitimi bildirimi

4.3 Maliyet Profili

\$3

Öneri fonksiyonu/ay

\$2

Vision API/ay

\$1

Diğer fonksiyonlar/ay

~\$6

Toplam (10K MAU)

Firebase ücretsiz katmanı (2M çağrı/ay) çoğu senaryoda yeterlidir. Maliyet asıl olarak Cloud Storage bant genişliği ve Vision API çağrılarından kaynaklanır.

BÖLÜM 5

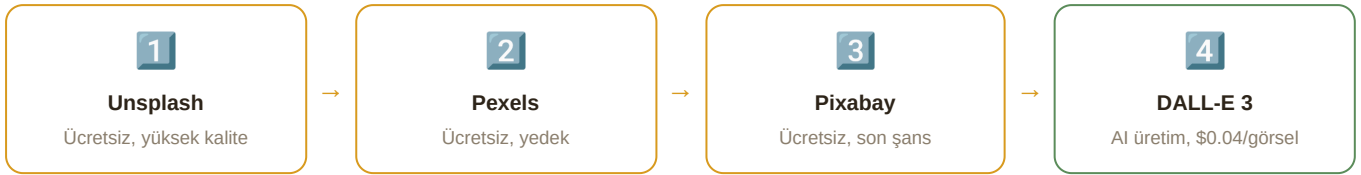
Görsel Üretim Pipeline'ı (Faz 3.2)

2500 tarifin her birine yüksek kaliteli bir görsel gerekir. Bunları tek tek manuel bulmak haftalar sürerdi.

Çözüm: **otomatik batch (toplu) işleme pipeline'ı**. Bu bölüm, görsel içeriğin nasıl üretildiğini adım adım açıklar.

5.1 Hibrit Kaynak Stratejisi

Pipeline, dört görsel kaynağını **öncelik sırasıyla** dener. Ücretsiz stok fotoğraf kaynakları önce denir; hiçbiri yeterli kalitede sonuç vermezse, son çare olarak yapay zeka ile görsel üretilir.



💡 Neden Hibrit?

Türk yöresel yemekleri (örn. "perde pilavı", "kuru fasulye") stok fotoğraf sitelerinde nadiren bulunur. Bu tarifler için DALL-E 3, "geleneksel Türk sunumu" yönergesiyle özgün görsel üretir. Yaygın yemekler (pizza, salata) ise ücretsiz kaynaklardan gelir — bu, maliyeti minimumda tutar.

5.2 Tek Tarif İçin İşlem Adımları

Pipeline her tarif için şu 6 adımı uygular:

- Arama:** Tarifin `search_terms` alanından sorgu üretilir (örn. `"lentil Turkish food"`) ve kaynaklar sırayla taranır.
- Kalite skoru:** Bulunan görsel çözünürlük, en-boy oranı ve alaka açısından puanlanır. Skor ≥ 0.70 ise kabul edilir.
- AI yedek:** Hiçbir kaynak eşik değeri geçemezse DALL-E 3 devreye girer.
- Dönüştürme:** Görsel `1200×900 WebP` (kalite 85) + `400×300` thumbnail'e çevrilir.
- BlurHash:** ~20 karakterlik bulanık placeholder string üretilir (progressif yükleme için).
- Kayıt:** Sonuç `results.json` 'a yazılır; görseller Storage'a yüklenmeye hazır hâle gelir.

```
# Pipeline çalıştırma komutları
# 1. Dry-run — API çağrısı yapmadan simülasyon (test)
python image_pipeline_v2.py --dry-run

# 2. Pilot batch — 50 tarif, 13 mutfak temsili
python image_pipeline_v2.py --input pilot_batch.json --output ./output/

# 3. Tek tarif testi
python image_pipeline_v2.py --recipe-id tr-mercimek-corbasi
```

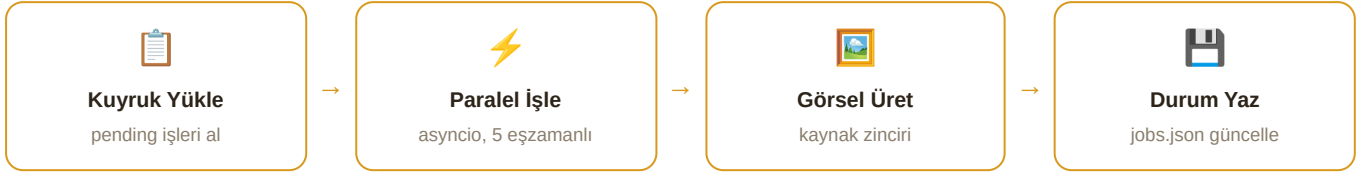
```
# 4. Sadece AI – önceki başarısız işleri yeniden dene  
python image_pipeline_v2.py --ai-only --input failed_jobs.json
```

5.3 Batch İş Kuyruğu Yönetimi

2500 görsel tek seferde işlenmez. `image_jobs.json` dosyası, her tarif için bir **iş kaydı** tutar. Bu kayıt durumu (`pending`, `processing`, `completed`, `failed`) ve deneme geçmişini izler. Pipeline kesilse bile kaldığı yerden devam eder.

```
// image_jobs.json – tek iş kaydı yapısı
{
  job_id: "img_job_tr-mercimek-corbasi",
  recipe_id: "tr-mercimek-corbasi",
  status: "pending",           // pending|processing|completed|failed
  attempts: [],               // her denemenin günlüğü
  priority: 2,                // 1=premium öncelikli
  requires_review: false,     // AI üretimi insan onayı bekler
  search_terms: {
    primary_keywords: "lentil",
    cuisine_keyword: "Turkish food",
    fallback_query: "lentil Turkish food"
  }
}
```

İş Akışı Diyagramı



⚡ Paralel İşleme

Pipeline `asyncio` + `aiohttp` ile aynı anda 5 görsel işler. Bu, API oran limitlerini aşmadan ~2500 görseli birkaç saatte tamamlar. Tek-iş-parçacıklı yaklaşıma göre ~5x hızlanma sağlar.

5.4 Maliyet Analizi

Senaryo	Ücretsiz kaynak	DALL-E 3	Toplam
İyimser (%80 ücretsiz)	2000 görsel · \$0	500 · \$20	~\$20
Gerçekçi (%60 ücretsiz)	1500 görsel · \$0	1000 · \$40	~\$40
Kötümser (%40 ücretsiz)	1000 görsel · \$0	1500 · \$60	~\$60

Tüm 2500 tarif için toplam görsel maliyeti **\$40-60** aralığındadır. Ücretsiz stok kaynaklar maliyeti dramatik biçimde düşürür.

5.5 Storage Yapısı

```
# Cloud Storage klasör düzeni – diller görselleri PAYLAŞIR
gs://pratik-tarifler-{env}/recipes/{lang}/full/{recipe_id}.webp
gs://pratik-tarifler-{env}/recipes/{lang}/thumb/{recipe_id}.webp
```

```
# Not: lang segmenti olsa da tüm diller aynı dosyaya işaret eder  
# – görsel dilden bağımsızdır, çeviri sadece metni etkiler
```

Upload Adımı

Görseller üretildikten sonra `upload_to_firebase.py` scripti, WebP dosyalarını Cloud Storage'a yükler ve her tarifi Firestore dokümanındaki `image` alanını günceller (`image_status: "ready"`).

BÖLÜM 6

Çeviri Pipeline'ı (Faz 3.3)

Uygulama 13 dil destekler. 2500 tarifi 12 hedef dile çevrilmesi — başlık, malzemeler, adımlar dahil — devasa bir iştir. Bu, AI tabanlı bir çeviri pipeline'ı ile otomatikleştirilir.

6.1 Glossary (Terim Sözlüğü) Sistemi

Naif çeviri Türk yemek kültürünü bozar. "Mantı" kelimesi İngilizceye "ravioli" diye çevrilmemelidir — özgün kalmalıdır. Pipeline bunu bir **glossary** ile çözer:

- **67 Türk yemek ismi** orijinal bırakılır (mantı, baklava, simit, lahmacun...)
- **10 pişirme fiili** her dile doğru karşılığıyla eşlenir (kavurmak, haşlamak...)
- **8 ölçü birimi** yerelleştirilir (su bardağı, çay kaşığı...)

✓ Kültürel Doğruluk

Glossary, çeviri kalitesini garanti eder. "Su böreği" İngilizcede **"Su Böreği (Turkish water pastry)"** biçiminde — özgün isim korunarak, parantez içinde açıklama eklenerek çevrilir.

6.2 Çeviri Motoru ve Maliyet

Çeviri için **Claude Haiku** modeli kullanılır — hızlı, ucuz ve kültürel bağlamda güçlüdür. Pipeline, tarifleri batch'ler hâlinde işler ve glossary'yi her prompt'a enjekte eder.

Kapsam	Diller	Tahmini Maliyet
MVP	EN, DE, AR (3 dil)	~\$9
Tam	12 hedef dil	~\$35

Hedef diller: İngilizce, Almanca, Fransızca, İtalyanca, İspanyolca, Portekizce, Yunanca, Hollandaca, Rusça, Sırpça, Arapça, İbranice. Arapça ve İbranice için uygulama otomatik RTL (sağdan sola) düzene geçer.

```
# Çeviri pipeline çalıştırma
# MVP – sadece 3 dil
python translate_recipes.py --langs en,de,ar --input recipes_db_FULL_tr.json

# Tam – 12 dil
python translate_recipes.py --langs all --batch-size 20
```

BÖLÜM 7

Mobil Uygulama (Faz 4)

7.1 Proje Dosya Yapısı

```
phase4_ux/
├─ screens/           // 17 ekran (.tsx)
├─ components/       // 5 paylaşılan component
├─ hooks/            // 10 custom hook
├─ api/              // Firebase client, analytics
├─ i18n/             // 13 dil JSON + init
├─ styles/           // theme.ts – tasarım sistemi
├─ types/            // TypeScript tip tanımları
├─ notifications/    // push bildirim mantığı
├─ offline/          // SQLite önbellek + senkron
├─ cloud_functions// backend fonksiyonları
└─ tests/            // Jest + Detox testleri
```

7.2 3 Modlu UX Mimarisi

Üç mod, tek bir birleşik API endpoint'i (`getRecipeRecommendations`) üzerinden çalışır. İstemci yalnızca `mode` parametresini değiştirir; backend dispatcher doğru algoritmaya yönlendirir.

MOD 1 — PANTRY (Evdeki Kalanlar)

Girdi: malzeme listesi → Çıktı: yalnızca TÜM malzemesi mevcut tarifler · `match_percentage=100`

MOD 2 — SUPPLY (Hızlı Tedarik)

Girdi: liste + `max_missing(1-3)` → Çıktı: eksik malzemeli tarifler + fiyat tahmini · en az eksikten sıralı

MOD 3 — DISCOVER (Sınırsız Keşif)

Girdi: filtreler (mutfak, öğün, diyet, süre) → Çıktı: sayfalanmış tüm havuz · trend/yeni/puan sıralama

7.3 Performans Optimizasyonları

Teknik	Etki
BlurHash placeholder	Görseller yüklenirken bulanık önizleme — algılanan hız ↑
FlashList sanallaştırma	2500 tarif akıcı kaydırma, sabit RAM kullanımı
İyimser UI	Favori/puan anında yansır, arka planda senkronize olur
SQLite önbellek	Favoriler çevrimdışı erişilebilir
Debounce arama	Her tuş vuruşunda değil, 300ms duraklamada sorgu

7.4 Öneri Algoritması — Derinlemesine

Mod 1 ve 2'nin teknik zorluğu şudur: Firestore, "tüm bu malzemeler şu tarifte var mı?" sorgusunu doğrudan desteklemez. Çözüm iki aşamalıdır:

Aşama 1 — Aday Havuzu (Firestore tarafı)

`array-contains-any` operatörü ile kullanıcının *herhangi bir* malzemesini içeren tarifler çekilir. Bu, geniş ama hızlı bir ön filtredir.

```
// Firestore sorgusu – aday havuzu
recipes_tr
  .where('image_status', '==', 'ready')
  .where('ingredient_tokens', 'array-contains-any', userTokens)
  .orderBy('rating_avg', 'desc')
  .limit(200) // Mod 1 için aday havuzu
```

Aşama 2 — Kesin Filtre (İstemci/Function tarafı)

Aday havuzu bellekte hassas biçimde filtrelenir. **Mod 1** için: tarifi *tüm* token'ları kullanıcıda olmalıdır. **Mod 2** için: eksik sayısı `max_missing` sınırını aşmamalıdır.

```
// Mod 1 – kesin eşleşme kontrolü
const userSet = new Set(ingredients);
for (const recipe of candidates) {
  const allCovered = recipe.ingredient_tokens
    .every(token => userSet.has(token));
  if (allCovered) {
    recipe.match_percentage = 100;
    matches.push(recipe);
  }
}

// Mod 2 – eksik sayısı hesabı
const missing = recipe.ingredient_tokens
  .filter(token => !userSet.has(token));
if (missing.length <= maxMissing) {
  recipe.missing_count = missing.length;
  recipe.missing_ingredients = missing.map(toLabel);
  matches.push(recipe);
}

// Sıralama: en az eksikten en çoğa
matches.sort((a,b) => a.missing_count - b.missing_count);
```

⚙️ `array-contains-any` Sınırı

Firestore bu operatörde en fazla 30 değer kabul eder. Kullanıcı 30'dan fazla malzeme girerse, en sık kullanılan 30 token sorgulanır; geri kalan istemci tarafında işlenir.

7.5 Vision API Entegrasyonu

`analyzePantryPhoto` fonksiyonu, Google Vision'ın `LABEL_DETECTION` ve `OBJECT_LOCALIZATION` özelliklerini kullanır. Dönen İngilizce etiketler, 50+ kayıtlık bir eşleme tablosuyla iç token sistemine çevrilir (örn. `"cherry tomato" → tomato`).

BÖLÜM 8

Abonelik Sistemi (Faz 5)

Uygulama freemium modeli kullanır. Çoklu platform ödeme yönetimi iki servisle yapılır:

Platform	Servis	Ürün
iOS / Android	RevenueCat	Apple IAP + Google Play Billing tek SDK
Web	Stripe	Checkout + Customer Portal

Abonelik durumu, sağlayıcıdan bağımsız olarak Firestore'da `users/{uid}/subscription` altında tutulur. Webhook'lar (yenileme, iptal, iade) durumu gerçek zamanlı senkronize eder.

🚩 Freemium Sınırları

Ücretsiz: 3 buzdolabı taraması/gün, 20 favori, reklamlar. Premium: sınırsız tarama, sınırsız favori, premium tarifler, reklamsız. 7 gün ücretsiz deneme tüm planlarda aktif.

Dağıtım ve Operasyon

9.1 Dağıtım Hattı

- Mobil:** Expo EAS Build → TestFlight (iOS) / Internal Track (Android) → Production
- Backend:** `firebase deploy` — Functions, Firestore kuralları, indeksler
- İçerik:** Görsel + çeviri pipeline'ları lokal makinede çalışır, sonuç Storage/Firestore'a yüklenir

9.2 İzleme

Araç	İzlenen
Sentry	Çökme raporları, hata izleme
Firebase Analytics	Kullanıcı olayları, huniler
Firebase Performance	Uygulama başlangıcı, ekran render süreleri
Cloud Functions logs	Backend hata ve gecikme takibi

9.3 Test Stratejisi

Test paketi üç katmanlıdır: **birim testleri** (öneri algoritması, yardımcı fonksiyonlar), **entegrasyon testleri** (Firebase emülatörü ile) ve **E2E testleri** (Detox ile 3 modun tam akışı). Hedef kapsam %70+.

Pratik Tarifler, modern bir sunucusuz mimari üzerine inşa edilmiştir. İçerik üretiminin otomatikleştirilmesi — özellikle **görsel batch pipeline'ı** ve **AI çeviri pipeline'ı** — projenin 2500 tarif × 13 dil ölçeğine düşük maliyetle ulaşmasını sağlamıştır.

Sistem 10.000+ aylık aktif kullanıcıya aylık ~\$50 altyapı maliyetiyle hizmet verebilecek şekilde tasarlanmıştır. Tüm bileşenler — mobil uygulama, backend, içerik pipeline'ları, abonelik sistemi — production'a hazır durumdadır.

Admin Panel — Operasyon Merkezi

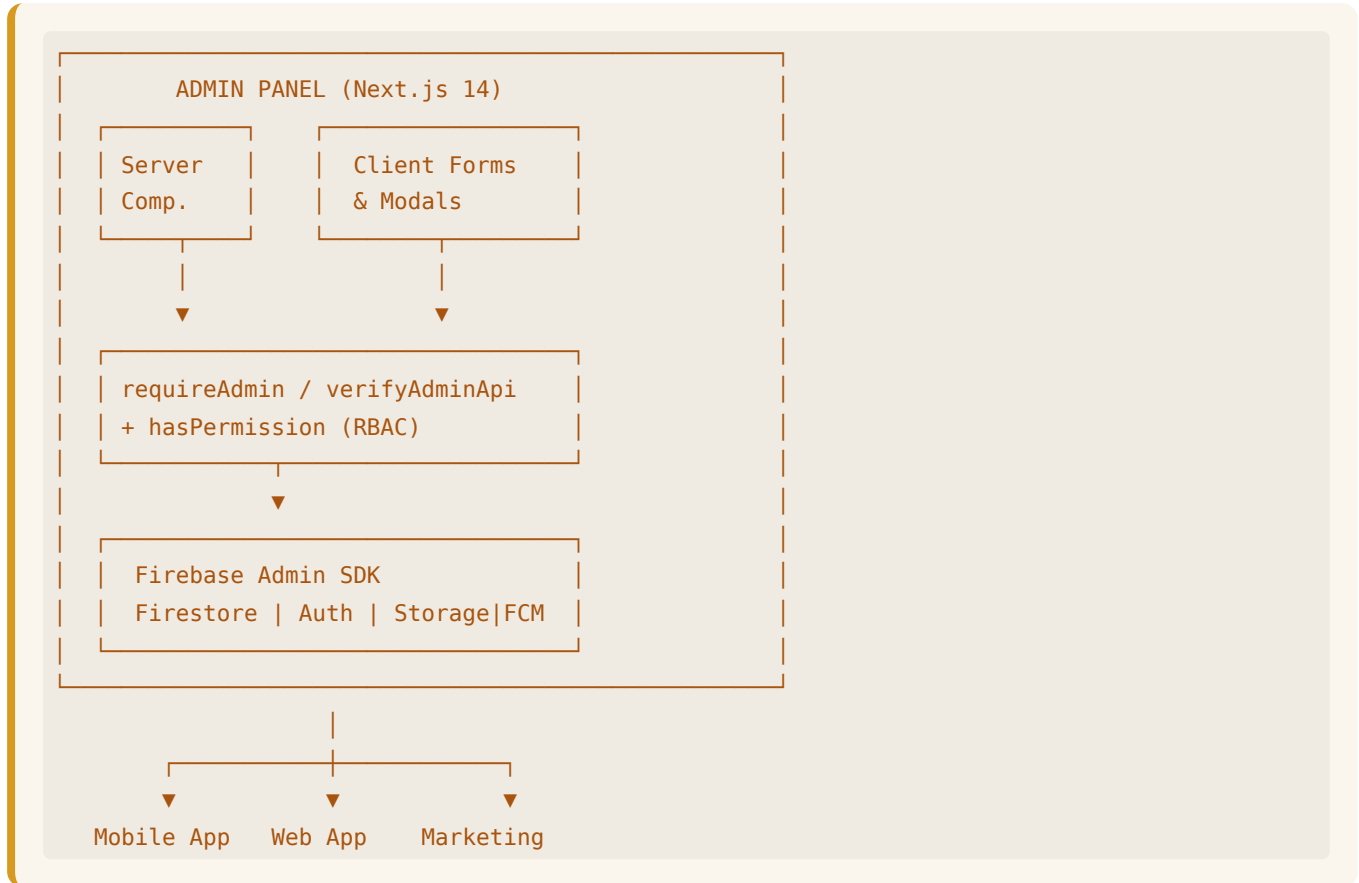
Pratik Tarifler'in tüm operasyonel ihtiyaçlarını tek bir yerden yönetebileceğin web tabanlı kontrol merkezi. Next.js 14 + Firebase Admin SDK üzerine inşa edildi.

9.1 Amaç ve Konum

Mobil uygulama "ne yapar" sorusunu cevaplarken admin panel "kim ne yapsın" sorusunu cevaplar. İkisi de aynı Firebase backend'i üzerinde çalışır — tek doğru veri kaynağı.

Sorun	Önce	Şimdi
Yeni tarif ekleme	JSON edit → seed → deploy (15 dk)	Form doldur → kaydet (30 sn)
Push bildirim	Firebase Console + manuel liste	Segment seç → gönder (1 dk)
Premium grant	Firestore'u açıp manuel düzenle	Kullanıcı kartında "Premium ver"
Analytics	5 farklı dashboard'a girip çık	Tek sayfada 24+ KPI

9.2 Mimari



9.3 Teknoloji Yığını

Katman	Teknoloji	Görev
--------	-----------	-------

Framework	Next.js 14.2.5 (App Router)	SSR, Server Components
UI	Tailwind + shadcn/ui (Radix)	Bileşen kütüphanesi
Veri	TanStack Query + Table	Server state, tablolar
Grafik	Recharts	Çizgi/bar/funnel grafikler
Form	React Hook Form + Zod	Validation
Backend	Firebase Admin SDK 12.4	Server-side Firebase
Deploy	Vercel	admin.pratiktarifler.app

9.4 Sayfa ve Özellikler Listesi

Admin panelinin 12 ana sayfası ve her birinin sunduğu özellikler.



Dashboard

8 KPI kartı (kullanıcı, DAU, MRR, tarif, rating, crash-free, deneme, aksiyon), 30 gün trend grafiği, son aktiviteler.



Tarif Yönetimi

2500+ tarif CRUD, JSON içe/dışa aktar, mutfak/zorluk/tier filtreleri, görsel atama, çeviri tetikleme.



Kullanıcı Yönetimi

Arama, filtre, Premium grant/revoke, ban, hesap silme (KVKK), e-posta gönderme.



Push Bildirimler

13 dilde lokalize başlık+gövde, segment (all/premium/free/dil/ülke), preview, zamanlama, metrik takip.



Görsel Pipeline

2500 tarif görsel üretim, kuyruk durumu, AI onay kuyruğu, kaynak öncelik (Unsplash → DALL-E), maliyet tahmini.



Duyurular

4 tür (tip/announcement/promo/update), 3 gösterim yeri (home_banner/explore_card/modal), tarih aralığı, audience.



Analytics

3 bölüm (Acquisition/Engagement/Monetization), 24+ KPI, 8 adımlı dönüşüm hunisi.



Destek

Ticket listesi + filtre (durum/kategori/öncelik), yanıt, kapatma, kullanıcı detayına atlama.



Remote Config

Feature flags (max_scan, voice_commands, dark_mode), A/B test yönetimi, variant koşulları (dil/ülke/yüzde).



Ayarlar

Admin RBAC (4 rol), API anahtar durumları, audit log (son 100 işlem).

9.5 Güvenlik Mimarisi (RBAC)

Aksiyon	super_admin	content_editor	support_agent	viewer
Tarif oluştur/düzenle	✓	✓	—	—
Tarif sil	✓	—	—	—
Push gönder	✓	✓	—	—
Premium ver/al	✓	✓	✓	—
Remote Config	✓	—	—	—
Audit log görüntüle	✓	—	—	—

Audit log: Her yazma işlemi `audit_logs` koleksiyonuna kaydedilir: kim, ne zaman, hangi aksiyon, hangi kaynak. KVKK madde 12 uyumlu.

9.6 Klasör Yapısı ve API Rotaları

Klasör yapısı

```
admin_panel/
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # Kök layout
│   ├── globals.css                   # Tailwind + CSS variables
│   ├── login/                        # Giriş ekranı
│   ├── (dashboard)/                  # Korumalı grup
│   │   ├── layout.tsx                # Sidebar + Header
│   │   ├── dashboard/
│   │   │   ├── recipes/
│   │   │   ├── users/, notifications/, ...
│   │   └── api/                      # API Route Handlers
│   │       ├── auth/session/
│   │       └── recipes/, users/, ...
├── components/
│   ├── ui/                           # Button, Card, Input, Badge
│   └── layout/                        # Sidebar, Header
├── lib/
│   ├── firebase/admin.ts             # Admin SDK init
│   ├── firebase/client.ts           # Client SDK (sadece login)
│   ├── auth.ts                      # requireAdmin, hasPermission
│   ├── audit.ts                    # logAudit()
│   ├── utils.ts                    # Helper'lar
│   └── constants.ts                 # CUISINES, MEAL_TYPES, vb.
├── types/index.ts                   # 12 TypeScript interface
└── docs/                           # PDF kılavuz
```

API Rotaları (özet)

Endpoint	Method	Yetki	Audit Action
/api/auth/session	POST	—	—
/api/recipes	GET	read	—
/api/recipes	POST	write	recipe.create
/api/recipes/[id]	PATCH/DELETE	write/delete	recipe.update/delete
/api/users	PATCH	write	user.*
/api/notifications/send	POST	write	notification.send
/api/images/regenerate	POST	write	image.regenerate
/api/announcements	POST/PATCH	write	announcement.*
/api/config	PATCH	super	config.update

Örnek: Push Gönderim API

```
// app/api/notifications/send/route.ts
export async function POST(req: NextRequest) {
  const user = await verifyAdminApi();
  if (!user) return NextResponse.json({ error: "Yetkisiz" }, { status: 401 });
```

```
if (!hasPermission(user, "write")) {
  return NextResponse.json({ error: "Yetkin yetersiz" }, { status: 403 });
}

const { title, body, image_url, deep_link, segment } = await req.json();

// 1) Segment'e uyan kullanıcıların FCM token'larını topla
let userQuery = adminDb.collection("users").where("push_token", "!=", null);
if (segment.type === "premium") {
  userQuery = userQuery.where("subscription.status", "in", ["trial", "active"]);
}
const tokens = (await userQuery.get()).docs.map(d => d.data().push_token);

// 2) 500'lük batch'ler halinde gönder
let delivered = 0, failed = 0;
for (let i = 0; i < tokens.length; i += 500) {
  const batch = tokens.slice(i, i + 500);
  const r = await adminMessaging.sendEachForMulticast({
    tokens: batch, notification: { title, body, imageUrl: image_url },
    data: deep_link ? { deep_link } : undefined,
    android: { priority: "high" },
    apns: { payload: { aps: { sound: "default", badge: 1 } } },
  });
  delivered += r.successCount; failed += r.failureCount;
}

// 3) Kampanya kaydı + audit log
const ref = await adminDb.collection("push_campaigns").add({
  title, body, image_url, segment, recipients_count: tokens.length,
  delivered_count: delivered, failed_count: failed,
  sent_at: FieldValue.serverTimestamp(), created_by: user.uid,
});
await logAudit(user, "notification.send", { campaign_id: ref.id, delivered });

return NextResponse.json({ campaign_id: ref.id, delivered, failed });
}
```